

用户管理系统—安全审计与漏洞修复报告

项目名称: user-management-security
审计日期: 2026-07-07
审计目标: Flask 用户信息管理平台
风险等级: 严重

1. 漏洞汇总

编号	漏洞名称	风险等级	描述
V-001	密码明文存储	严重	密码以明文形式直接存储在USERS字典中
V-002	密码明文显示	严重	登录后将密码明文传给模板并展示在页面上
V-003	硬编码密钥	高危	secret_key 为弱密钥“dev-key-2025”
V-004	调试信息泄露	中危	HTML 注释中泄露默认管理员账号密码
V-005	无会话安全配置	中危	缺少 session 安全选项
V-006	无 CSRF 保护	中危	登录表单无 CSRF Token
V-007	暴力破解防护缺失	中危	登录接口无频率限制
V-008	调试模式开启	中危	debug=True 暴露敏感信息
V-009	账号固定风险	低危	登录后未重新生成 session
V-010	无 HTTPS	低危	明文传输密码
V-011	日志泄露敏感信息	低危	Debugger PIN 暴露在终端输出中

2. 高危漏洞详解

V-001: 密码明文存储

位置: app.py 第 9-20 行, USERS 字典

问题代码:

```
USERS = {  
    "admin": {  
        "password": "admin123", # 明文存储  
    },  
    "alice": {  
        "password": "alice2025", # 明文存储  
    }  
}
```

风险: - 数据库泄露导致所有密码直接暴露 - 用户习惯复用密码, 可导致撞库攻击 - 内部人员可直接获取所有用户密码

修复方案:

```
from werkzeug.security import generate_password_hash, check_password_hash  
  
# 存储时加密  
"password": generate_password_hash("admin123")  
  
# 验证时比对哈希  
if user and check_password_hash(user["password"], password):  
    # 密码正确
```

V-002: 密码明文显示

位置: templates/index.html 第 16 行

风险: - 登录后密码直接展示在页面上 - 旁人路过屏幕即可看到密码 - 违反网络安全法和等保要求

修复方案: 删除模板中的密码显示行, 后端返回用户信息时过滤掉 password 字段。

V-003: 硬编码弱密钥

位置: app.py 第 5 行

问题代码:

```
app.secret_key = "dev-key-2025"
```

风险： - 密钥强度极低，可被暴力破解 - session 可被伪造，导致任意用户登录

修复方案：

```
import os
app.secret_key = os.environ.get('SECRET_KEY', os.urandom(32).hex())
```

3. 中危漏洞详解

V-004：调试信息泄露

位置： templates/login.html 第 1 行

```
<!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
```

风险： 查看网页源码即可获得管理员账号密码。

修复方案： 删除该 HTML 注释。

V-005：无会话安全配置

缺失配置：

```
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['SESSION_COOKIE_SECURE'] = True
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(hours=2)
```

V-006：无 CSRF 保护

风险： 攻击者可构造恶意页面诱导用户提交表单。

修复方案： 集成 Flask-WTF 的 CSRF 保护：

```
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)
```

V-007：暴力破解防护缺失

风险： 攻击者可通过脚本无限尝试密码。

修复方案： 集成 Flask-Limiter 限制登录频率：

```
from flask_limiter import Limiter
limiter = Limiter(get_remote_address, app=app)
```

V-008: 开启调试模式

风险: 调试器可执行任意 Python 代码。

修复方案: 生产环境关闭 debug, 使用 debug=False。

4. 低危漏洞详解

编号	漏洞名称	说明
V-009	账号固定	登录后未调用 session.regenerate()
V-010	无 HTTPS	密码在网络上明文传输, 局域网内可被嗅探
V-011	日志泄露	Debugger PIN 在终端可见

5. 修复方案汇总

编号	修复措施	风险等级变化
V-001	使用 generate_password_hash 哈希存储密码	严重 -> 已修复
V-002	删除模板中的密码显示行	严重 -> 已修复
V-003	使用环境变量加载密钥	高危 -> 已修复
V-004	删除 HTML 注释	中危 -> 已修复
V-005	配置完整的 session 安全选项	中危 -> 已修复
V-006	集成 Flask-WTF 的 CSRF 保护	中危 -> 已修复
V-007	集成 Flask-Limiter 限制请求频率	中危 -> 已修复
V-008	生产环境关闭 debug	中危 -> 已修复
V-009	登录成功时调用 session.regenerate()	低危 -> 已修复

6. 安全建议

短期 (立即修复)

1. 删除密码明文显示
2. 更换 secret_key 为强随机密钥
3. 关闭 debug 模式
4. 删除 HTML 注释中的调试信息

中期（1-2 天内）

1. 集成 CSRF 保护
2. 添加登录频率限制
3. 配置 session 安全选项
4. 实现 session 超时

长期（架构层面）

1. 使用数据库替代字典存储用户信息
2. 集成 OAuth2 / SSO 认证
3. 全站 HTTPS
4. 添加安全日志审计系统
5. 定期进行安全渗透测试

附录：OWASP Top 10 映射

漏洞编号	OWASP 分类
V-001、V-002	A02:2021 —加密机制失效
V-003、V-005	A05:2021 —安全配置错误
V-004	A01:2021 —失效的访问控制
V-006	A04:2021 —不安全的设计
V-007	A01:2021 —失效的访问控制
V-008	A05:2021 —安全配置错误
V-010	A02:2021 —加密机制失效（传输层）

报告编写： Claude Code AI Assistant

报告版本： v1.0

免责声明： 本报告仅用于安全学习和测试目的。